ASSIGNMENT 3

Database Design Analysis

Redundancy, anomalies, functional dependencies and design improvements

1. Objective and Scope

The finalized design of the Online Shopping System uses eleven tables: **Customers, Addresses, Categories, Products, Shopping_Cart, Cart_Items, Orders, Order_Items, Payments, Shipments** and **Reviews**. This report examines that design for data redundancy and for insertion, deletion and update anomalies, identifies the functional dependencies (FDs) that exist between attributes, explains how a poor design produces anomalies, and proposes improvements. The improvements are carried forward into the implementation in Assignments 5–9.

2. Functional Dependencies in the Design

A functional dependency **X → Y** means that the value of X uniquely determines the value of Y. The FDs below are derived from the business rules of the application (for example, "an e-mail address identifies exactly one customer" and "a postal PIN code belongs to exactly one city and state"). **CK** = candidate key, **NF** = highest normal form satisfied by the original design.

Table	Candidate key(s)	Functional dependencies	NF (original)
Customers	customer_id; email	customer_id → first_name, last_name, email, phone, password_hash, registration_date email → customer_id	BCNF
Addresses	address_id	address_id → customer_id, address_line, city, state, pincode, address_type **pincode → city, state** (non-key determines non-key)	**2NF** – transitive dependency
Categories	category_id; category_name	category_id → category_name, description category_name → category_id	BCNF
Products	product_id; (product_name, brand)	product_id → category_id, product_name, description, price, stock_quantity, brand (product_name, brand) → product_id	BCNF (brand determines nothing)
Shopping_Cart	cart_id; customer_id	cart_id → customer_id, created_date customer_id → cart_id (one active cart per customer)	BCNF
Cart_Items	cart_item_id; (cart_id, product_id)	cart_item_id → cart_id, product_id, quantity (cart_id, product_id) → cart_item_id, quantity	BCNF
Orders	order_id	order_id → customer_id, address_id, order_date, order_status, total_amount **address_id → customer_id** (an address belongs to one customer) order_id → total_amount = Σ(quantity × unit_price) (derived)	**2NF** – transitive dependency + derived column
Order_Items	order_item_id; (order_id, product_id)	order_item_id → order_id, product_id, quantity, unit_price (order_id, product_id) → order_item_id, quantity, unit_price	BCNF
Payments	payment_id; order_id	payment_id → order_id, payment_method, payment_status, payment_date, amount_paid order_id → payment_id	BCNF
Shipments	shipment_id; order_id; tracking_number	shipment_id → order_id, courier_name, tracking_number, shipped_date, expected_delivery_date, shipment_status tracking_number → shipment_id	BCNF
Reviews	review_id; (customer_id, product_id)	review_id → product_id, customer_id, rating, comment, review_date (customer_id, product_id) → review_id (one review per product per customer)	BCNF

Reading the result. Nine of the eleven tables are already in BCNF – the surrogate primary keys plus the splitting of Cart_Items and Order_Items out of their parents did the heavy lifting. Two tables contain a *transitive dependency* (a non-key attribute determining another non-key attribute): **Addresses** (pincode → city, state) and **Orders** (address_id → customer_id). In addition, **Orders.total_amount** is a derived value. These are the weak points examined in the rest of this report.

3. Data Redundancy Analysis

#	Redundancy found	Where	Harmful?	Decision
R1	City and state text repeated for every address in the same PIN code	Addresses.city, Addresses.state	Yes – causes all three anomalies	Remove: new Pincodes table
R2	customer_id stored in Orders although it is implied by address_id	Orders.customer_id	Potentially – customer and address could disagree	Keep for convenience, **enforce with composite FK**
R3	Order total can be recomputed from the line items	Orders.total_amount	Can go stale if items change	Keep as controlled denormalization; verify with a query
R4	Payment amount overlaps the order total	Payments.amount_paid	No – differs for Failed / Pending / COD	Keep (different meaning)
R5	Delivered state appears in both order and shipment	Orders.order_status, Shipments.shipment_status	Mildly – can drift apart	Keep; keep in sync in the application transaction
R6	Price copied from Products into each order line	Order_Items.unit_price	No – it is a historical snapshot	Keep (required for correct billing history)
R7	Brand name typed repeatedly as free text	Products.brand	Low – spelling variants possible	Considered; Brands table optional, not adopted

4. Insertion, Deletion and Update Anomalies

4.1 Definitions

- **Insertion anomaly** – a fact cannot be stored because an unrelated fact is missing (e.g. a new product cannot be saved until someone orders it).
- **Deletion anomaly** – deleting one fact accidentally destroys another fact stored in the same row (e.g. deleting the only order for a product also deletes the product's name and price).
- **Update anomaly** – one real-world fact is stored in many rows, so changing it in only some of them leaves the database contradicting itself.

4.2 How poor design leads to anomalies

Anomalies appear when **one table stores facts about more than one entity**. If customer details, product details and order details share a row, then every order line repeats the customer and product facts (redundancy). Each repeated copy is an independent value, so an update that reaches only some copies causes inconsistency; the row cannot exist unless *all* its parts exist (insertion anomaly); and the row is the only place some fact is recorded, so removing the row removes the fact (deletion anomaly). Formally, this happens when a table has **partial dependencies** (a part of a composite key determines a non-key attribute) or **transitive dependencies** (a non-key attribute determines another non-key attribute). Normalization removes these dependencies by moving each fact to the one table whose key determines it, so that every fact is stored exactly once.

4.3 Demonstration 1 – A single “flat” order sheet (what poor design looks like)

To make the effect visible, the order data of customer #3 is copied into one wide, un-normalized table *flat_orders* in a scratch database. This is the design a beginner might use (one spreadsheet-style table) and it is **not** part of the final design.

Setup (scratch database)

```
DROP DATABASE IF EXISTS anomaly_demo;
CREATE DATABASE anomaly_demo;
USE anomaly_demo;
CREATE TABLE flat_orders (
  order_id INT NOT NULL, product_id INT NOT NULL, order_date DATE,
  customer_name VARCHAR(80), customer_email VARCHAR(100), customer_phone VARCHAR(15),
  product_name VARCHAR(100), category_name VARCHAR(50),
  unit_price DECIMAL(10,2), quantity INT,
  PRIMARY KEY (order_id, product_id));
INSERT INTO flat_orders
SELECT o.order_id, i.product_id, DATE(o.order_date),
       CONCAT(c.first_name,' ',c.last_name), c.email, c.phone,
       p.product_name, cat.category_name, i.unit_price, i.quantity
FROM online_shopping.Orders o
JOIN online_shopping.Customers c    ON c.customer_id = o.customer_id
```

```
JOIN online_shopping.Order_Items i  ON i.order_id    = o.order_id
JOIN online_shopping.Products p     ON p.product_id  = i.product_id
JOIN online_shopping.Categories cat ON cat.category_id = p.category_id
WHERE o.order_id IN (4, 8, 20);
```

Redundancy – the same customer is stored on every row

SQL query

```
SELECT order_id AS ord, product_id AS pid, customer_name, customer_phone, LEFT(product_name,26) AS product, unit_price, quantity AS
qty
FROM flat_orders ORDER BY order_id, product_id;
```

Output

```
+-----+-----+---------------+----------------+---------------------------+------------+------+
| ord | pid | customer_name | customer_phone | product                   | unit_price | qty  |
+-----+-----+---------------+----------------+---------------------------+------------+------+
|   4 |   2 | Rohan Mehta   | 7093414184     | Rockerz 450 Bluetooth Head |    1619.00 |    2 |
|   8 |   7 | Rohan Mehta   | 7093414184     | Men's Formal Shirt         |    1499.00 |    1 |
|   8 |  10 | Rohan Mehta   | 7093414184     | Revolution 7 Running Shoes |    3995.00 |    1 |
|   8 |  18 | Rohan Mehta   | 7093414184     | Atomic Habits              |     499.00 |    3 |
|  20 |   3 | Rohan Mehta   | 7093414184     | 25W Fast Charger           |    1199.00 |    1 |
|  20 |  14 | Rohan Mehta   | 7093414184     | Induction Cooktop 2000W    |    2499.00 |    1 |
|  20 |  18 | Rohan Mehta   | 7093414184     | Atomic Habits              |     499.00 |    1 |
+-----+-----+---------------+----------------+---------------------------+------------+------+
```

Explanation: Customer #3's name and phone number appear seven times, once per order line. Product details would repeat in the same way if a product were ordered again.

Insertion anomaly

Adding a new product that nobody has ordered yet

SQL query

```
INSERT INTO flat_orders (product_id, product_name, category_name, unit_price)
VALUES (26, 'Smart LED Bulb', 'Home & Kitchen', 499.00);
```

Output

```
ERROR 1364 (HY000): Field 'order_id' doesn't have a default value
```

Explanation: The insert is rejected because the primary key (order_id, product_id) needs an order. A product cannot be added to the catalogue until it is sold – a classic insertion anomaly.

Update anomaly

Customer changes phone number – but only one row is updated

SQL query

```
UPDATE flat_orders SET customer_phone = '9800000000' WHERE order_id = 4;
SELECT DISTINCT customer_email, customer_phone FROM flat_orders;
```

Output

```
+----------------------+----------------+
| customer_email       | customer_phone |
+----------------------+----------------+
| rohan.mehta@gmail.com | 9800000000     |
| rohan.mehta@gmail.com | 7093414184     |
+----------------------+----------------+
```

Explanation: The same customer now has two different phone numbers in the same table. The database no longer knows which is correct. In a normalized design the phone number lives in one row of Customers, so this cannot happen.

Deletion anomaly

Cancelling and purging the only order that contained a product

SQL query

```
SET @pid = (SELECT product_id FROM flat_orders WHERE order_id = 4);
SELECT product_id, LEFT(product_name,26) AS product, category_name, unit_price FROM flat_orders WHERE product_id = @pid;
DELETE FROM flat_orders WHERE order_id = 4;
SELECT COUNT(*) AS rows_left_for_product FROM flat_orders WHERE product_id = @pid;
```

Output

```
+------------+---------------------------+---------------+------------+
| product_id | product                   | category_name | unit_price |
+------------+---------------------------+---------------+------------+
|          2 | Rockerz 450 Bluetooth Head | Electronics   |    1619.00 |
```

```
+------------+---------------------------+--------------+------------+
+-----------------------+
| rows_left_for_product |
+-----------------------+
|                     0 |
+-----------------------+
```

Explanation: Before the delete the product's name, category and price are known; afterwards **0 rows** remain, so all information about the product was lost along with the order.

4.4 Demonstration 2 – The real weakness in the original Addresses table

The original Addresses table stored *city* and *state* next to *pincode*. Because **pincode → city, state**, this is a transitive dependency (address_id → pincode → city, state). The table *addresses_v1* reproduces that original structure with the project's real address data.

Setup

```
USE anomaly_demo;
CREATE TABLE addresses_v1 (
  address_id INT PRIMARY KEY, customer_id INT NOT NULL, address_line VARCHAR(150) NOT NULL,
  city VARCHAR(50), state VARCHAR(50), pincode VARCHAR(10), address_type VARCHAR(10));
INSERT INTO addresses_v1
SELECT a.address_id, a.customer_id, a.address_line, p.city, p.state, a.pincode, a.address_type
FROM online_shopping.Addresses a JOIN online_shopping.Pincodes p ON p.pincode = a.pincode;
```

Redundancy – city/state repeated for every address in the same PIN code

SQL query

```
SELECT pincode, city, state, COUNT(*) AS rows_repeating_this_fact
FROM addresses_v1 GROUP BY pincode, city, state HAVING COUNT(*) > 1;
```

Output

```
+---------+-----------+-------------+--------------------------+
| pincode | city      | state       | rows_repeating_this_fact |
+---------+-----------+-------------+--------------------------+
| 110016  | New Delhi | Delhi       |                        2 |
| 400050  | Mumbai    | Maharashtra |                        3 |
| 411045  | Pune      | Maharashtra |                        2 |
| 560001  | Bengaluru | Karnataka   |                        2 |
| 560034  | Bengaluru | Karnataka   |                        2 |
| 560066  | Bengaluru | Karnataka   |                        2 |
+---------+-----------+-------------+--------------------------+
```

Explanation: Each listed PIN code's city and state are stored once per address. Every extra address adds another copy of the same fact.

Update anomaly – one row is spelt differently

SQL query

```
UPDATE addresses_v1 SET city = 'Bangalore' WHERE address_id = 1;
SELECT address_id, pincode, city FROM addresses_v1 WHERE pincode = '560034';
```

Output

```
+------------+---------+-----------+
| address_id | pincode | city      |
+------------+---------+-----------+
|          1 | 560034  | Bangalore |
|         14 | 560034  | Bengaluru |
+------------+---------+-----------+
```

Explanation: Addresses 1 and 14 share PIN code 560034 but now show two different city names. Renaming a city correctly would require updating every matching row.

Insertion anomaly – recording a PIN code before any customer lives there

SQL query

```
INSERT INTO addresses_v1 (pincode, city, state) VALUES ('641001', 'Coimbatore', 'Tamil Nadu');
```

Output

```
ERROR 1364 (HY000): Field 'address_id' doesn't have a default value
```

Explanation: The mapping 641001 → Coimbatore cannot be stored on its own, because an address row (with its own key and customer) must exist first.

Deletion anomaly – removing the last address in a PIN code

SQL query

```
DELETE FROM addresses_v1 WHERE address_id = 22;
SELECT * FROM addresses_v1 WHERE pincode = '600040';
```

Output

```
(statement executed successfully, no rows returned)
```

Explanation: Address 22 was the only address in PIN code 600040. After it is deleted, the fact "600040 belongs to Chennai, Tamil Nadu" has disappeared from the database.

4.5 Redundancy that is deliberate: Orders.total_amount and Order_Items.unit_price

Check that the stored order total matches the line items (should return 0 mismatches)

SQL query

```
SELECT COUNT(*) AS mismatched_orders FROM (
  SELECT o.order_id FROM Orders o JOIN Order_Items i ON i.order_id = o.order_id
  GROUP BY o.order_id, o.total_amount
  HAVING o.total_amount <> SUM(i.quantity * i.unit_price)) x;
```

Output

```
+-------------------+
| mismatched_orders |
+-------------------+
|                 0 |
+-------------------+
```

Explanation: The stored totals currently agree with the line items. The risk is that a later change to an Order_Items row without updating Orders would silently break this – shown next.

What a stale total looks like (change is rolled back afterwards)

SQL query

```
START TRANSACTION;
UPDATE Order_Items SET quantity = quantity + 1 WHERE order_item_id = 1;
SELECT o.order_id, o.total_amount AS stored_total, SUM(i.quantity*i.unit_price) AS recomputed_total
FROM Orders o JOIN Order_Items i ON i.order_id = o.order_id WHERE o.order_id = 1 GROUP BY o.order_id, o.total_amount;
ROLLBACK;
```

Output

```
+----------+--------------+------------------+
| order_id | stored_total | recomputed_total |
+----------+--------------+------------------+
|        1 |     24379.00 |         25878.00 |
+----------+--------------+------------------+
```

Explanation: After the item changed, the stored and recomputed totals disagree – an update anomaly. For this reason total_amount is treated as a controlled copy: it is written only inside the checkout transaction, and the verification query above is used as an integrity audit.

unit_price is NOT redundancy – it records the price at the time of purchase

SQL query

```
SELECT LEFT(p.product_name,26) AS product, p.price AS current_price, i.unit_price AS price_when_ordered, i.order_id
FROM Order_Items i JOIN Products p ON p.product_id = i.product_id
WHERE i.unit_price <> p.price ORDER BY i.order_id LIMIT 6;
```

Output

```
+----------------------------+---------------+--------------------+----------+
| product                    | current_price | price_when_ordered | order_id |
+----------------------------+---------------+--------------------+----------+
| Redmi Note 13 5G (8GB/128G |      17999.00 |           19439.00 |        1 |
| M331 Silent Plus Wireless  |        899.00 |             971.00 |        1 |
| 25W Fast Charger           |       1199.00 |            1295.00 |        2 |
| 25W Fast Charger           |       1199.00 |            1295.00 |        3 |
| Rockerz 450 Bluetooth Head |       1499.00 |            1619.00 |        4 |
| M331 Silent Plus Wireless  |        899.00 |             971.00 |        5 |
+----------------------------+---------------+--------------------+----------+
```

Explanation: When the catalogue price changes, old orders must still show what the customer actually paid. Therefore product_id does *not* determine unit_price, and no transitive dependency exists here.

5. Proposed Improvements

#	Improvement	Problem it solves	Result
1	Create **Pincodes(pincode PK, city, state)**; Addresses keeps only a FK *pincode*	R1: transitive dependency pincode → city, state; all three anomalies	Addresses and Pincodes both in BCNF
2	Add UNIQUE(address_id, customer_id) on Addresses and a **composite FK Orders(address_id, customer_id) → Addresses**	R2: order could point to another customer's address	Redundant Orders.customer_id can never disagree with the address owner
3	UNIQUE constraints on every candidate key: Customers.email, Categories.category_name, Products(product_name, brand), Cart_Items(cart_id, product_id), Order_Items(order_id, product_id), Reviews(customer_id, product_id), Shipments.tracking_number, and the 1:1 links (cart.customer_id, payments.order_id, shipments.order_id)	Duplicate rows (same product twice in a cart, two reviews by one customer, duplicate e-mail)	Every FD listed in Section 2 is enforced by the DBMS
4	CHECK constraints on price, stock, quantity, rating and status/method columns; DEFAULT values for status and dates	Invalid or missing values	Domain integrity
5	Keep Orders.total_amount and Order_Items.unit_price, documented as controlled denormalization with a verification query	R3 / R6: needed for history and fast reporting	Redundancy is deliberate and auditable
6	Considered but not adopted: Brands table; order-status history table	R7 / R5: low benefit for the project scope	Can be added later without redesign

5.1 Proof: the improved design removes the anomalies

The same four scenarios from Demonstration 2 are repeated on the real, improved tables. Every change is wrapped in a transaction and rolled back, so the data is left untouched.

Insertion, update and deletion on the improved Pincodes / Addresses tables

SQL query

```
START TRANSACTION;
-- Insertion: PIN code can exist without any address
INSERT INTO Pincodes VALUES ('641001', 'Coimbatore', 'Tamil Nadu');
-- Update: one row changed -> every address in 560034 shows the new name
UPDATE Pincodes SET city = 'Bengaluru Urban' WHERE pincode = '560034';
SELECT a.address_id, a.pincode, p.city FROM Addresses a
JOIN Pincodes p ON p.pincode = a.pincode WHERE a.pincode = '560034';
-- Deletion: removing the last address in 600040 keeps the PIN code fact
DELETE FROM Addresses WHERE address_id = 22;
SELECT * FROM Pincodes WHERE pincode IN ('641001', '600040');
ROLLBACK;
```

Output

```
+------------+---------+-----------------+
| address_id | pincode | city            |
+------------+---------+-----------------+
|          1 | 560034  | Bengaluru Urban |
|         14 | 560034  | Bengaluru Urban |
+------------+---------+-----------------+
+---------+------------+------------+
| pincode | city       | state      |
+---------+------------+------------+
| 600040  | Chennai    | Tamil Nadu |
| 641001  | Coimbatore | Tamil Nadu |
+---------+------------+------------+
```

Explanation: (1) The Coimbatore PIN code is stored with no address – no insertion anomaly. (2) A single UPDATE corrects the city for all addresses – no update anomaly. (3) Deleting address 22 leaves PIN code 600040 (Chennai) in Pincodes – no deletion anomaly.

6. Revised (Finalized) Design and Normal-Form Check

Table	Primary key	Foreign keys	Highest NF now
Customers	customer_id	–	BCNF
Pincodes *(new)*	pincode	–	BCNF

Table	Primary key	Foreign keys	Highest NF now
Addresses	address_id	customer_id → Customers; pincode → Pincodes	BCNF
Categories	category_id	–	BCNF
Products	product_id	category_id → Categories	BCNF
Shopping_Cart	cart_id	customer_id → Customers (UNIQUE)	BCNF
Cart_Items	cart_item_id	cart_id → Shopping_Cart; product_id → Products	BCNF
Orders	order_id	(address_id, customer_id) → Addresses; customer_id → Customers	3NF*
Order_Items	order_item_id	order_id → Orders; product_id → Products	BCNF
Payments	payment_id	order_id → Orders (UNIQUE)	BCNF
Shipments	shipment_id	order_id → Orders (UNIQUE)	BCNF
Reviews	review_id	product_id → Products; customer_id → Customers	BCNF

* Orders keeps the intentional, database-enforced copy of customer_id and the derived total_amount described in Sections 3 and 5; the composite foreign key guarantees the copy can never contradict Addresses.

7. Conclusion

The original design was already well structured: only Addresses (pincode → city, state) and Orders (address_id → customer_id, derived total) deviated from third normal form. The live demonstrations show that mixing several entities in one table creates redundancy and leads directly to insertion, deletion and update anomalies. By adding the Pincodes table, enforcing every candidate key with UNIQUE constraints, adding the composite foreign key and domain CHECK constraints, and documenting the deliberate denormalizations, the final 12-table design stores each fact in exactly one place and lets the DBMS itself reject inconsistent data.